Lab 4 - Advanced Algorithms: Recursion, Backtracking, and GUI Programming

This lab introduces arrays, recursion, GUI programming, and advanced algorithmic thinking through six progressive tasks. Students will learn two-dimensional array manipulation, recursive method design, event-driven programming, and backtracking algorithms. The exercises progress from maze pathfinding through Game of Life simulation, GUI calculator development, game AI implementation, fractal generation, and culminate with an optional Sudoku solver, emphasizing problem decomposition, visual programming, and advanced data structure manipulation.

Contents

Memorize

Submission Deadline

Deadline to upload the solutions for all tasks is Saturday, 11:59 pm before the lab date.

General Information

The following tasks are to be worked on in fixed teams of two. Each team member must be able to explain all solutions. Please submit only one solution for each team of two. The submission must be a PDF file in our Moodle room with the name and matriculation number. Solutions must be in digital format with intermediate steps and detailed explanations (no handwritten scans). You can use any tool or drawing program of your choice to create the diagrams. If you have questions or need support, use the forum in our Moodle room and help each other.

1. Task 1: Maze Pathfinding with Recursion

Create a recursive pathfinding algorithm to help a mouse navigate through a labyrinth to reach cheese. This task introduces two-dimensional arrays, recursion, and backtracking algorithms.

A poor, hungry mouse sits in the upper left corner of a labyrinth (see sketch) and wants to reach a piece of cheese located in the lower right corner of the labyrinth. It can enter all non-hatched fields, but only via an edge shared by two adjacent fields. Help the mouse reach the cheese. Write a recursive method in Java that shows the mouse a path to the cheese.

> 🔥 **Tip**
>
> Your method must try for every possible field to find a path to the cheese via each of the four neighboring fields.

Your program should:

1. Represent the labyrinth using a 2D character array
2. Implement a recursive `findPath(int row, int col)` method
3. Use backtracking when paths lead to dead ends
4. Mark visited cells to prevent infinite loops
5. Display the final path through the maze

1.1. Preparation

Represent the labyrinth in a two-dimensional array. Use a method that checks all four directions and calls itself recursively to find the path. Tip: Use a marking character to mark the path that the mouse has taken, as the mouse should not go backwards. If you want, you can recreate the labyrinth with graphical methods and also enter the mouse's path to the cheese. A modification of the labyrinth or a larger number of fields is also possible.

1.2. Assistance

2D Array representation:

```java
public class MazeSolver {
    private char[][] maze = {
        {'#', '#', '#', '#', '#', '#', '#'},
        {'#', ' ', ' ', '#', ' ', ' ', '#'},
        {'#', ' ', '#', '#', ' ', '#', '#'},
        {'#', ' ', ' ', ' ', ' ', '#', '#'},
        {'#', '#', '#', ' ', '#', ' ', '#'},
        {'#', ' ', ' ', ' ', ' ', ' ', '#'},
        {'#', '#', '#', '#', '#', '#', '#'}
    };

    private int startRow = 1, startCol = 1;
    private int endRow = 5, endCol = 5;
}
```

1.3. Solution

```java
public class MazeSolver {
  private char[][] maze;
  private int startRow, startCol;
  private int endRow, endCol;
  private static final char WALL = '#';
  private static final char PATH = ' ';
```

```
7       private static final char VISITED = '.';
8       private static final char SOLUTION = '*';
9
10      public MazeSolver() {
11        // Initialize maze based on the lab image
12        this.maze = new char[][] {
13            {'#', '#', '#', '#', '#', '#', '#', '#', '#', '#'},
14            {'#', ' ', ' ', ' ', '#', ' ', ' ', ' ', ' ', '#'},
15            {'#', ' ', '#', ' ', '#', ' ', '#', '#', ' ', '#'},
16            {'#', ' ', '#', ' ', ' ', ' ', ' ', '#', ' ', '#'},
17            {'#', ' ', '#', '#', '#', '#', ' ', '#', ' ', '#'},
18            {'#', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', '#'},
19            {'#', '#', '#', '#', '#', '#', '#', '#', '#', '#'}
20        };
21
22        // Mouse starts at top-left corner
23        this.startRow = 1;
24        this.startCol = 1;
25
26        // Cheese is at bottom-right corner
27        this.endRow = 5;
28        this.endCol = 8;
29      }
30
31      // Display the current maze state
32      public void displayMaze() {
33        for (int i = 0; i < maze.length; i++) {
34          for (int j = 0; j < maze[i].length; j++) {
35            System.out.print(maze[i][j] + " ");
36          }
37          System.out.println();
38        }
39        System.out.println();
40      }
41
42      // Check if a position is valid and can be visited
43      private boolean isValid(int row, int col) {
44        // Check boundaries
45        if (row < 0 || row >= maze.length || col < 0 || col >= maze[0].length) {
46          return false;
47        }
48
49        // Check if it's not a wall and not already visited
```

```
50      return maze[row][col] == PATH || maze[row][col] == SOLUTION;
51    }
52
53    // Recursive pathfinding method
54    public boolean findPath(int row, int col) {
55      // Base case: reached the cheese!
56      if (row == endRow && col == endCol) {
57        maze[row][col] = SOLUTION;
58        return true;
59      }
60
61      // Check if current position is valid
62      if (!isValid(row, col)) {
63        return false;
64      }
65
66      // Mark current position as part of solution path
67      maze[row][col] = SOLUTION;
68
69      // Try all four directions: up, down, left, right
70      // Try moving up
71      if (findPath(row - 1, col)) {
72        return true;
73      }
74
75      // Try moving down
76      if (findPath(row + 1, col)) {
77        return true;
78      }
79
80      // Try moving left
81      if (findPath(row, col - 1)) {
82        return true;
83      }
84
85      // Try moving right
86      if (findPath(row, col + 1)) {
87        return true;
88      }
89
90      // Backtrack: this path doesn't lead to cheese
91      maze[row][col] = VISITED;
92      return false;
```

```
93    }
94
95    // Solve the maze and display the solution
96    public void solve() {
97      System.out.println("=== Maze Pathfinding ===\n");
98      System.out.println("Initial Maze:");
99      displayMaze();
100
101     System.out.println("Finding path from (" + startRow + ", " + startCol +
102                        ") to (" + endRow + ", " + endCol + ")...\n");
103
104     if (findPath(startRow, startCol)) {
105       System.out.println("Path found! (* = solution path, . = tried but
    failed)");
106       displayMaze();
107     } else {
108       System.out.println("No path found!");
109     }
110   }
111
112   public static void main(String[] args) {
113     MazeSolver solver = new MazeSolver();
114     solver.solve();
115   }
116 }
```

2. Task 2: Conway's Game of Life Simulator

2.1. Preparation

Before implementing the solution, you must:

1. Design a class structure for the Game of Life simulator
2. Create a UML class diagram showing:
   - Class name (GameOfLife)
   - All private attributes (grid, rows, cols, generation counter, etc.)
   - All public methods with their parameters and return types (printGrid, nextGeneration, countNeighbors, etc.)
   - Initialization methods for different patterns (initGlider, initRandom, etc.)
   - Constructor(s)
3. Consider what data structures you need (2D boolean array for the grid)
4. Transfer your implementation into this class structure
5. Only after completing the UML diagram should you begin coding

This preparation step ensures you think about the simulation architecture and state management before writing code.

2.2. Task

Create a simulation of Conway's Game of Life, a cellular automaton that demonstrates how complex patterns emerge from simple rules. This task introduces 2D array manipulation, simulation loops, and pattern evolution.

Conway's Game of Life is a zero-player game where cells on a grid live, die, or reproduce based on their neighbors:

- A live cell with 2-3 live neighbors survives
- A dead cell with exactly 3 live neighbors becomes alive
- All other cells die or stay dead

Your program should:

1. Create a 2D boolean array to represent the grid (true = alive, false = dead)
2. Initialize the grid with a pattern (e.g., glider, blinker, or random)
3. Implement `countNeighbors(int row, int col)` to count live neighbors
4. Create `nextGeneration()` to compute the next state
5. Display each generation and pause between updates
6. Run for a specified number of generations or until stable

2.3. Requirements

- Use a 2D array to represent the grid (minimum 20x20)
- Implement the four Game of Life rules correctly
- Display each generation in a readable format
- Handle edge cases at grid boundaries
- Allow user to choose initial pattern or use random seed

2.4. Assistance

Grid initialization and display:

```java
public class GameOfLife {
    private boolean[][] grid;
    private int rows, cols;

    public GameOfLife(int rows, int cols) {
        this.rows = rows;
        this.cols = cols;
        grid = new boolean[rows][cols];
    }

    public void printGrid() {
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                System.out.print(grid[i][j] ? "\u2588 " : "\u00b7 ");
            }
            System.out.println();
        }
        System.out.println();
    }

    // Initialize with glider pattern
    public void initGlider(int startRow, int startCol) {
        grid[startRow][startCol + 1] = true;
        grid[startRow + 1][startCol + 2] = true;
        grid[startRow + 2][startCol] = true;
        grid[startRow + 2][startCol + 1] = true;
        grid[startRow + 2][startCol + 2] = true;
    }
}
```

Counting neighbors:

```java
private int countNeighbors(int row, int col) {
    int count = 0;
    for (int i = -1; i <= 1; i++) {
        for (int j = -1; j <= 1; j++) {
            if (i == 0 && j == 0) continue;
            int newRow = row + i;
            int newCol = col + j;
            if (newRow >= 0 && newRow < rows &&
                newCol >= 0 && newCol < cols &&
                grid[newRow][newCol]) {
                count++;
            }
```

```
13          }
14      }
15      return count;
16  }
```

2.5. Solution

```java
public class GameOfLife {
    private boolean[][] grid;
    private int rows;
    private int cols;
    private int generation;

    // Constructor
    public GameOfLife(int rows, int cols) {
        this.rows = rows;
        this.cols = cols;
        this.grid = new boolean[rows][cols];
        this.generation = 0;
    }

    // Print the current grid
    public void printGrid() {
        System.out.println("Generation " + generation + ":");
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                System.out.print(grid[i][j] ? "\u2588 " : "\u00b7 ");
            }
            System.out.println();
        }
        System.out.println();
    }

    // Initialize with a glider pattern
    public void initGlider(int startRow, int startCol) {
        grid[startRow][startCol + 1] = true;
        grid[startRow + 1][startCol + 2] = true;
        grid[startRow + 2][startCol] = true;
        grid[startRow + 2][startCol + 1] = true;
        grid[startRow + 2][startCol + 2] = true;
    }

    // Initialize with a blinker pattern (oscillator)
    public void initBlinker(int startRow, int startCol) {
```

```
38      grid[startRow][startCol] = true;
39      grid[startRow][startCol + 1] = true;
40      grid[startRow][startCol + 2] = true;
41    }
42
43    // Initialize with random pattern
44    public void initRandom(double density) {
45      for (int i = 0; i < rows; i++) {
46        for (int j = 0; j < cols; j++) {
47          grid[i][j] = Math.random() < density;
48        }
49      }
50    }
51
52    // Count live neighbors for a cell
53    private int countNeighbors(int row, int col) {
54      int count = 0;
55
56      // Check all 8 surrounding cells
57      for (int i = -1; i <= 1; i++) {
58        for (int j = -1; j <= 1; j++) {
59          // Skip the cell itself
60          if (i == 0 && j == 0) continue;
61
62          int newRow = row + i;
63          int newCol = col + j;
64
65          // Check boundaries and count live neighbors
66          if (newRow >= 0 && newRow < rows &&
67              newCol >= 0 && newCol < cols &&
68              grid[newRow][newCol]) {
69            count++;
70          }
71        }
72      }
73
74      return count;
75    }
76
77    // Calculate next generation
78    public void nextGeneration() {
79      boolean[][] newGrid = new boolean[rows][cols];
80
```

```
81      // Apply Conway's rules to each cell
82      for (int i = 0; i < rows; i++) {
83        for (int j = 0; j < cols; j++) {
84          int neighbors = countNeighbors(i, j);
85
86          if (grid[i][j]) {
87            // Cell is alive
88            // Rule 1 & 3: Cell survives if it has 2 or 3 neighbors
89            newGrid[i][j] = (neighbors == 2 || neighbors == 3);
90          } else {
91            // Cell is dead
92            // Rule 4: Cell becomes alive if it has exactly 3 neighbors
93            newGrid[i][j] = (neighbors == 3);
94          }
95        }
96      }
97
98      // Update grid
99      grid = newGrid;
100     generation++;
101   }
102
103   // Count total living cells
104   public int countLivingCells() {
105     int count = 0;
106     for (int i = 0; i < rows; i++) {
107       for (int j = 0; j < cols; j++) {
108         if (grid[i][j]) count++;
109       }
110     }
111     return count;
112   }
113
114   // Run simulation for specified number of generations
115   public void simulate(int generations, int delay) {
116     for (int i = 0; i < generations; i++) {
117       printGrid();
118       System.out.println("Living cells: " + countLivingCells());
119
120       // Pause between generations
121       try {
122         Thread.sleep(delay);
123       } catch (InterruptedException e) {
```

```
124             e.printStackTrace();
125           }
126
127           nextGeneration();
128         }
129
130         // Print final state
131         printGrid();
132         System.out.println("Living cells: " + countLivingCells());
133       }
134
135       public static void main(String[] args) {
136         // Create a 25x25 grid
137         GameOfLife game = new GameOfLife(25, 25);
138
139         System.out.println("=== Conway's Game of Life ===\n");
140
141         // Initialize with a glider
142         System.out.println("Initializing with Glider pattern...\n");
143         game.initGlider(5, 5);
144
145         // Add a blinker
146         game.initBlinker(10, 15);
147
148         // Run simulation for 30 generations with 500ms delay
149         game.simulate(30, 500);
150       }
151     }
```

3. Task 3: Simple GUI Calculator

3.1. Preparation

Before implementing the solution, you must:

1. Design a class structure for the GUI calculator
2. Create a UML class diagram showing:
   - Class name (Calculator) extending JFrame and implementing ActionListener
   - All private attributes (display field, button arrays, operands, current operation, etc.)
   - All public methods with their parameters and return types
   - The actionPerformed method (from ActionListener interface)
   - Helper methods for calculations (add, subtract, multiply, divide)
   - Constructor that sets up the GUI
3. Consider the GUI component hierarchy (which panels contain which components)
4. Plan your event handling strategy (how button clicks update state)
5. Transfer your implementation into this class structure
6. Only after completing the UML diagram should you begin coding

This preparation step ensures you think about GUI architecture, state management, and event-driven design before writing code.

3.2. Task

Create a graphical calculator application using Java Swing. This task introduces GUI programming, event handling, and building interactive applications.

Build a calculator with a graphical interface that performs basic arithmetic operations. This task helps you understand event-driven programming and user interface design.

Your program should:

1. Create a window with number buttons (0-9), operation buttons (+, -, *, /), equals, and clear
2. Display a text field showing the current input and result
3. Handle button clicks to build expressions
4. Evaluate expressions when equals is pressed
5. Clear the display when the clear button is pressed
6. Handle decimal numbers and basic error cases (division by zero)

3.3. Requirements

- Use Java Swing components (JFrame, JButton, JTextField, JPanel)
- Use GridLayout or BorderLayout for component arrangement
- Implement ActionListener for button events
- Display results in a readable format
- Handle basic error cases gracefully

3.4. Assistance

Basic GUI structure:

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
```

```
4
5   public class Calculator extends JFrame implements ActionListener {
6       private JTextField display;
7       private JButton[] numberButtons;
8       private JButton[] operationButtons;
9       private double num1, num2, result;
10      private char operation;
11
12      public Calculator() {
13          setTitle("Calculator");
14          setSize(400, 500);
15          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
16          setLayout(new BorderLayout());
17
18          // Display field
19          display = new JTextField();
20          display.setEditable(false);
21          display.setFont(new Font("Arial", Font.BOLD, 24));
22          add(display, BorderLayout.NORTH);
23
24          // Button panel
25          JPanel buttonPanel = new JPanel();
26          buttonPanel.setLayout(new GridLayout(4, 4, 5, 5));
27
28          // Initialize buttons here
29          numberButtons = new JButton[10];
30          for (int i = 0; i < 10; i++) {
31              numberButtons[i] = new JButton(String.valueOf(i));
32              numberButtons[i].addActionListener(this);
33              numberButtons[i].setFont(new Font("Arial", Font.PLAIN, 20));
34          }
35
36          add(buttonPanel, BorderLayout.CENTER);
37          setVisible(true);
38      }
39
40      @Override
41      public void actionPerformed(ActionEvent e) {
42          // Handle button clicks
43          String command = e.getActionCommand();
44          // Implement logic here
45      }
46
```

```
47      public static void main(String[] args) {
48          new Calculator();
49      }
50  }
```

3.5. Solution

Java

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Calculator extends JFrame implements ActionListener {
  private JTextField display;
  private JButton[] numberButtons;
  private JButton[] operationButtons;
  private JButton addButton, subButton, mulButton, divButton;
  private JButton decimalButton, equalsButton, clearButton, deleteButton;
  private double num1 = 0, num2 = 0, result = 0;
  private char operator;

  public Calculator() {
    // Frame setup
    setTitle("Simple Calculator");
    setSize(420, 550);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLayout(null);
    setResizable(false);

    // Display field
    display = new JTextField();
    display.setBounds(50, 25, 300, 50);
    display.setFont(new Font("Arial", Font.BOLD, 24));
    display.setEditable(false);
    display.setHorizontalAlignment(JTextField.RIGHT);
    add(display);

    // Create number buttons (0-9)
    numberButtons = new JButton[10];
    for (int i = 0; i < 10; i++) {
      numberButtons[i] = new JButton(String.valueOf(i));
      numberButtons[i].addActionListener(this);
      numberButtons[i].setFont(new Font("Arial", Font.BOLD, 18));
      numberButtons[i].setFocusable(false);
    }
```

```
38
39      // Create operation buttons
40      addButton = new JButton("+");
41      subButton = new JButton("-");
42      mulButton = new JButton("*");
43      divButton = new JButton("/");
44      decimalButton = new JButton(".");
45      equalsButton = new JButton("=");
46      clearButton = new JButton("C");
47      deleteButton = new JButton("Del");
48
49      operationButtons = new JButton[] {
50          addButton, subButton, mulButton, divButton,
51          decimalButton, equalsButton, clearButton, deleteButton
52      };
53
54      // Add action listeners and formatting to operation buttons
55      for (JButton button : operationButtons) {
56        button.addActionListener(this);
57        button.setFont(new Font("Arial", Font.BOLD, 18));
58        button.setFocusable(false);
59      }
60
61      // Position delete and clear buttons
62      deleteButton.setBounds(50, 90, 145, 50);
63      clearButton.setBounds(205, 90, 145, 50);
64
65      // Create button panel with GridLayout
66      JPanel buttonPanel = new JPanel();
67      buttonPanel.setBounds(50, 150, 300, 300);
68      buttonPanel.setLayout(new GridLayout(4, 4, 10, 10));
69
70      // Add buttons to panel in calculator layout
71      buttonPanel.add(numberButtons[7]);
72      buttonPanel.add(numberButtons[8]);
73      buttonPanel.add(numberButtons[9]);
74      buttonPanel.add(divButton);
75
76      buttonPanel.add(numberButtons[4]);
77      buttonPanel.add(numberButtons[5]);
78      buttonPanel.add(numberButtons[6]);
79      buttonPanel.add(mulButton);
80
```

```
81        buttonPanel.add(numberButtons[1]);
82        buttonPanel.add(numberButtons[2]);
83        buttonPanel.add(numberButtons[3]);
84        buttonPanel.add(subButton);
85
86        buttonPanel.add(decimalButton);
87        buttonPanel.add(numberButtons[0]);
88        buttonPanel.add(equalsButton);
89        buttonPanel.add(addButton);
90
91        add(buttonPanel);
92        add(deleteButton);
93        add(clearButton);
94
95        setVisible(true);
96      }
97
98      @Override
99      public void actionPerformed(ActionEvent e) {
100       // Handle number button clicks
101       for (int i = 0; i < 10; i++) {
102         if (e.getSource() == numberButtons[i]) {
103           display.setText(display.getText() + i);
104         }
105       }
106
107       // Handle decimal button
108       if (e.getSource() == decimalButton) {
109         if (!display.getText().contains(".")) {
110           display.setText(display.getText() + ".");
111         }
112       }
113
114       // Handle operation buttons
115       if (e.getSource() == addButton) {
116         num1 = Double.parseDouble(display.getText());
117         operator = '+';
118         display.setText("");
119       }
120
121       if (e.getSource() == subButton) {
122         num1 = Double.parseDouble(display.getText());
123         operator = '-';
```

```
124         display.setText("");
125       }
126
127       if (e.getSource() == mulButton) {
128         num1 = Double.parseDouble(display.getText());
129         operator = '*';
130         display.setText("");
131       }
132
133       if (e.getSource() == divButton) {
134         num1 = Double.parseDouble(display.getText());
135         operator = '/';
136         display.setText("");
137       }
138
139       // Handle equals button
140       if (e.getSource() == equalsButton) {
141         num2 = Double.parseDouble(display.getText());
142
143         switch (operator) {
144           case '+':
145             result = num1 + num2;
146             break;
147           case '-':
148             result = num1 - num2;
149             break;
150           case '*':
151             result = num1 * num2;
152             break;
153           case '/':
154             if (num2 != 0) {
155               result = num1 / num2;
156             } else {
157               display.setText("Error");
158               return;
159             }
160             break;
161         }
162
163         display.setText(String.valueOf(result));
164         num1 = result;
165       }
166
```

```
167     // Handle clear button
168     if (e.getSource() == clearButton) {
169       display.setText("");
170       num1 = 0;
171       num2 = 0;
172       result = 0;
173     }
174
175     // Handle delete button
176     if (e.getSource() == deleteButton) {
177       String currentText = display.getText();
178       if (currentText.length() > 0) {
179         display.setText(currentText.substring(0, currentText.length() - 1));
180       }
181     }
182   }
183
184   public static void main(String[] args) {
185     // Run on Event Dispatch Thread
186     SwingUtilities.invokeLater(() -> new Calculator());
187   }
188 }
```

4. Task 4: Tic-Tac-Toe with AI Opponent

Create a Tic-Tac-Toe game with a computer opponent. This task combines 2D arrays, game logic, and basic AI using the minimax algorithm or simple heuristics.

Build a playable Tic-Tac-Toe game where a human player competes against the computer. The computer should make intelligent moves to challenge the player.

Your program should:

1. Use a 3x3 2D array to represent the game board
2. Display the board after each move
3. Allow the human player to choose positions
4. Implement computer AI that makes strategic moves
5. Check for win conditions (rows, columns, diagonals)
6. Detect draw conditions when the board is full
7. Allow replay after game ends

4.1. Requirements

- Use a 2D character array for the board
- Validate user input (position must be empty and valid)
- Implement win detection for all 8 possible winning lines
- Computer should block player wins when possible
- Display clear game status (whose turn, winner, draw)

4.2. Assistance

Game board structure:

```java
public class TicTacToe {
    private char[][] board;
    private static final char EMPTY = ' ';
    private static final char PLAYER = 'X';
    private static final char COMPUTER = 'O';

    public TicTacToe() {
        board = new char[3][3];
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                board[i][j] = EMPTY;
            }
        }
    }

    public void printBoard() {
        System.out.println("  0 1 2");
        for (int i = 0; i < 3; i++) {
            System.out.print(i + " ");
            for (int j = 0; j < 3; j++) {
```

```
21                 System.out.print(board[i][j]);
22                 if (j < 2) System.out.print("|");
23             }
24             System.out.println();
25             if (i < 2) System.out.println("  -----");
26         }
27         System.out.println();
28     }
29
30     public boolean checkWin(char player) {
31         // Check rows
32         for (int i = 0; i < 3; i++) {
33             if (board[i][0] == player && board[i][1] == player &&
34                 board[i][2] == player) return true;
35         }
36
37         // Check columns
38         for (int j = 0; j < 3; j++) {
39             if (board[0][j] == player && board[1][j] == player &&
40                 board[2][j] == player) return true;
41         }
42
43         // Check diagonals
44         if (board[0][0] == player && board[1][1] == player &&
45             board[2][2] == player) return true;
46         if (board[0][2] == player && board[1][1] == player &&
47             board[2][0] == player) return true;
48
49         return false;
50     }
51
52     // Simple AI: Find winning move, block opponent, or choose random
53     public void computerMove() {
54         // Try to win
55         for (int i = 0; i < 3; i++) {
56             for (int j = 0; j < 3; j++) {
57                 if (board[i][j] == EMPTY) {
58                     board[i][j] = COMPUTER;
59                     if (checkWin(COMPUTER)) return;
60                     board[i][j] = EMPTY;
61                 }
62             }
63         }
```

```
64
65          // Try to block player
66          for (int i = 0; i < 3; i++) {
67              for (int j = 0; j < 3; j++) {
68                  if (board[i][j] == EMPTY) {
69                      board[i][j] = PLAYER;
70                      if (checkWin(PLAYER)) {
71                          board[i][j] = COMPUTER;
72                          return;
73                      }
74                      board[i][j] = EMPTY;
75                  }
76              }
77          }
78
79          // Choose center if available
80          if (board[1][1] == EMPTY) {
81              board[1][1] = COMPUTER;
82              return;
83          }
84
85          // Choose random available position
86          // Implementation here
87      }
88  }
```

4.3. Solution

Java

```
1   import java.util.Random;
2   import java.util.Scanner;
3
4   public class TicTacToe {
5     private char[][] board;
6     private static final char EMPTY = ' ';
7     private static final char PLAYER = 'X';
8     private static final char COMPUTER = 'O';
9     private Random random;
10
11    public TicTacToe() {
12      board = new char[3][3];
13      random = new Random();
14      // Initialize empty board
15      for (int i = 0; i < 3; i++) {
16        for (int j = 0; j < 3; j++) {
```

```
17          board[i][j] = EMPTY;
18        }
19      }
20    }
21
22    // Print the current board
23    public void printBoard() {
24      System.out.println("\n  0 1 2");
25      for (int i = 0; i < 3; i++) {
26        System.out.print(i + " ");
27        for (int j = 0; j < 3; j++) {
28          System.out.print(board[i][j]);
29          if (j < 2) System.out.print("|");
30        }
31        System.out.println();
32        if (i < 2) System.out.println("  -----");
33      }
34      System.out.println();
35    }
36
37    // Check if a position is valid and empty
38    public boolean isValidMove(int row, int col) {
39      return row >= 0 && row < 3 && col >= 0 && col < 3 && board[row][col] ==
EMPTY;
40    }
41
42    // Make a move
43    public void makeMove(int row, int col, char player) {
44      board[row][col] = player;
45    }
46
47    // Check if someone won
48    public boolean checkWin(char player) {
49      // Check rows
50      for (int i = 0; i < 3; i++) {
51        if (board[i][0] == player && board[i][1] == player && board[i][2] ==
player) {
52          return true;
53        }
54      }
55
56      // Check columns
57      for (int j = 0; j < 3; j++) {
```

```
58        if (board[0][j] == player && board[1][j] == player && board[2][j] ==
    player) {
59          return true;
60        }
61      }
62
63      // Check diagonals
64      if (board[0][0] == player && board[1][1] == player && board[2][2] ==
    player) {
65        return true;
66      }
67      if (board[0][2] == player && board[1][1] == player && board[2][0] ==
    player) {
68        return true;
69      }
70
71      return false;
72    }
73
74    // Check if board is full (draw)
75    public boolean isBoardFull() {
76      for (int i = 0; i < 3; i++) {
77        for (int j = 0; j < 3; j++) {
78          if (board[i][j] == EMPTY) {
79            return false;
80          }
81        }
82      }
83      return true;
84    }
85
86    // Computer makes a move using simple AI
87    public void computerMove() {
88      // Strategy 1: Try to win
89      for (int i = 0; i < 3; i++) {
90        for (int j = 0; j < 3; j++) {
91          if (board[i][j] == EMPTY) {
92            board[i][j] = COMPUTER;
93            if (checkWin(COMPUTER)) {
94              return; // Winning move found
95            }
96            board[i][j] = EMPTY; // Undo test move
97          }
```

```
98          }
99        }
100
101       // Strategy 2: Block player from winning
102       for (int i = 0; i < 3; i++) {
103         for (int j = 0; j < 3; j++) {
104           if (board[i][j] == EMPTY) {
105             board[i][j] = PLAYER;
106             if (checkWin(PLAYER)) {
107               board[i][j] = COMPUTER; // Block the player
108               return;
109             }
110             board[i][j] = EMPTY; // Undo test move
111           }
112         }
113       }
114
115       // Strategy 3: Take center if available
116       if (board[1][1] == EMPTY) {
117         board[1][1] = COMPUTER;
118         return;
119       }
120
121       // Strategy 4: Take a corner
122       int[][] corners = {{0, 0}, {0, 2}, {2, 0}, {2, 2}};
123       for (int[] corner : corners) {
124         if (board[corner[0]][corner[1]] == EMPTY) {
125           board[corner[0]][corner[1]] = COMPUTER;
126           return;
127         }
128       }
129
130       // Strategy 5: Take any available position
131       for (int i = 0; i < 3; i++) {
132         for (int j = 0; j < 3; j++) {
133           if (board[i][j] == EMPTY) {
134             board[i][j] = COMPUTER;
135             return;
136           }
137         }
138       }
139     }
140
```

```
141   // Play the game
142   public void play() {
143     Scanner scanner = new Scanner(System.in);
144     boolean gameOver = false;
145
146     System.out.println("=== Tic-Tac-Toe ===");
147     System.out.println("You are X, Computer is O");
148     printBoard();
149
150     while (!gameOver) {
151       // Player's turn
152       boolean validMove = false;
153       while (!validMove) {
154         System.out.print("Enter row (0-2): ");
155         int row = scanner.nextInt();
156         System.out.print("Enter column (0-2): ");
157         int col = scanner.nextInt();
158
159         if (isValidMove(row, col)) {
160           makeMove(row, col, PLAYER);
161           validMove = true;
162         } else {
163           System.out.println("Invalid move! Try again.");
164         }
165       }
166
167       printBoard();
168
169       // Check if player won
170       if (checkWin(PLAYER)) {
171         System.out.println("Congratulations! You win!");
172         gameOver = true;
173         break;
174       }
175
176       // Check for draw
177       if (isBoardFull()) {
178         System.out.println("It's a draw!");
179         gameOver = true;
180         break;
181       }
182
183       // Computer's turn
```

```
184         System.out.println("Computer's turn...");
185         computerMove();
186         printBoard();
187 
188         // Check if computer won
189         if (checkWin(COMPUTER)) {
190           System.out.println("Computer wins! Better luck next time.");
191           gameOver = true;
192           break;
193         }
194 
195         // Check for draw
196         if (isBoardFull()) {
197           System.out.println("It's a draw!");
198           gameOver = true;
199           break;
200         }
201       }
202 
203       scanner.close();
204     }
205 
206     public static void main(String[] args) {
207       TicTacToe game = new TicTacToe();
208       game.play();
209     }
210 }
```

5. Task 5: Recursive Fractal Tree Generator

Create a program that draws fractal trees using recursion. This task demonstrates recursive drawing, coordinate geometry, and visual pattern generation.

Implement a fractal tree generator that uses recursion to create beautiful branching patterns. Each branch spawns two smaller branches at angles, creating a tree-like structure.

Your program should:

1. Use Java graphics (JPanel, Graphics2D) to draw lines
2. Implement a recursive `drawBranch()` method
3. Start with a trunk and recursively draw smaller branches
4. Decrease branch length by a factor (e.g., 0.7) at each level
5. Split each branch into two branches at angles (e.g., ±30 degrees)
6. Stop recursion when branches become too small (base case)
7. Allow user to adjust parameters (angle, length factor, depth)

5.1. Requirements

- Use recursive method for drawing branches
- Apply trigonometry to calculate branch endpoints
- Implement proper base case to stop recursion
- Create a visually appealing tree structure
- Allow customization of tree parameters

5.2. Assistance

Basic fractal tree structure:

```java
import javax.swing.*;
import java.awt.*;

public class FractalTree extends JPanel {
    private int maxDepth = 10;

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2d = (Graphics2D) g;
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                             RenderingHints.VALUE_ANTIALIAS_ON);

        // Start drawing from bottom center
        int startX = getWidth() / 2;
        int startY = getHeight() - 50;
        int length = 100;

        drawBranch(g2d, startX, startY, length, -90, maxDepth);
    }
```

```
21
22      private void drawBranch(Graphics2D g2d, int x1, int y1,
23                              double length, double angle, int depth) {
24          if (depth == 0 || length < 2) return;
25
26          // Calculate end point of current branch
27          int x2 = x1 + (int)(length * Math.cos(Math.toRadians(angle)));
28          int y2 = y1 + (int)(length * Math.sin(Math.toRadians(angle)));
29
30          // Draw the branch
31          g2d.setColor(new Color(139, 69, 19)); // Brown
32          g2d.drawLine(x1, y1, x2, y2);
33
34          // Recursively draw two smaller branches
35          double newLength = length * 0.7;
36          drawBranch(g2d, x2, y2, newLength, angle - 25, depth - 1);
37          drawBranch(g2d, x2, y2, newLength, angle + 25, depth - 1);
38      }
39
40      public static void main(String[] args) {
41          JFrame frame = new JFrame("Fractal Tree");
42          FractalTree tree = new FractalTree();
43          frame.add(tree);
44          frame.setSize(800, 600);
45          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
46          frame.setVisible(true);
47      }
48  }
```

Extending with user controls:

```
1  // Add sliders or input fields to control:                    Java
2  // - maxDepth (recursion depth)
3  // - angle spread (branch angle)
4  // - lengthFactor (how much branches shrink)
5  // - initial trunk length
```

5.3. Solution

```
1    import javax.swing.*;                                       Java
2    import java.awt.*;
3
4    public class FractalTree extends JPanel {
5      private int maxDepth = 10;
6      private double angleSpread = 25.0; // degrees
```

```
7     private double lengthFactor = 0.7;
8
9     public FractalTree() {
10      setPreferredSize(new Dimension(800, 600));
11      setBackground(Color.WHITE);
12    }
13
14    @Override
15    protected void paintComponent(Graphics g) {
16      super.paintComponent(g);
17      Graphics2D g2d = (Graphics2D) g;
18
19      // Enable anti-aliasing for smooth lines
20      g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
21                           RenderingHints.VALUE_ANTIALIAS_ON);
22
23      // Start drawing from bottom center
24      int startX = getWidth() / 2;
25      int startY = getHeight() - 50;
26      int trunkLength = 120;
27
28      // Draw the tree starting with trunk going upward (-90 degrees)
29      drawBranch(g2d, startX, startY, trunkLength, -90, maxDepth);
30    }
31
32    /**
33     * Recursively draw a branch
34     * @param g2d Graphics context
35     * @param x1 Starting x coordinate
36     * @param y1 Starting y coordinate
37     * @param length Length of the branch
38     * @param angle Angle in degrees (0 = right, -90 = up)
39     * @param depth Remaining recursion depth
40     */
41    private void drawBranch(Graphics2D g2d, int x1, int y1,
42                            double length, double angle, int depth) {
43      // Base case: stop when depth reaches 0 or branch too small
44      if (depth == 0 || length < 2) {
45        return;
46      }
47
48      // Calculate end point of current branch using trigonometry
49      int x2 = x1 + (int)(length * Math.cos(Math.toRadians(angle)));
```

```
50      int y2 = y1 + (int)(length * Math.sin(Math.toRadians(angle)));
51
52      // Set color based on depth (brown for trunk, green for leaves)
53      if (depth > 3) {
54        // Brown for thicker branches
55        int brownShade = 139 - (maxDepth - depth) * 10;
56        brownShade = Math.max(50, Math.min(139, brownShade));
57        g2d.setColor(new Color(brownShade, 69, 19));
58      } else {
59        // Green for thinner branches (leaves)
60        g2d.setColor(new Color(34, 139, 34));
61      }
62
63      // Set stroke width based on depth
64      int strokeWidth = Math.max(1, depth / 2);
65      g2d.setStroke(new BasicStroke(strokeWidth));
66
67      // Draw the branch
68      g2d.drawLine(x1, y1, x2, y2);
69
70      // Calculate new length for child branches
71      double newLength = length * lengthFactor;
72
73      // Recursively draw left branch (angle - angleSpread)
74      drawBranch(g2d, x2, y2, newLength, angle - angleSpread, depth - 1);
75
76      // Recursively draw right branch (angle + angleSpread)
77      drawBranch(g2d, x2, y2, newLength, angle + angleSpread, depth - 1);
78    }
79
80    // Setters for parameters
81    public void setMaxDepth(int maxDepth) {
82      this.maxDepth = maxDepth;
83      repaint();
84    }
85
86    public void setAngleSpread(double angleSpread) {
87      this.angleSpread = angleSpread;
88      repaint();
89    }
90
91    public void setLengthFactor(double lengthFactor) {
92      this.lengthFactor = lengthFactor;
```

```
93        repaint();
94      }
95
96      public static void main(String[] args) {
97        SwingUtilities.invokeLater(() -> {
98          JFrame frame = new JFrame("Recursive Fractal Tree");
99          FractalTree tree = new FractalTree();
100
101         // Create control panel
102         JPanel controlPanel = new JPanel();
103         controlPanel.setLayout(new FlowLayout());
104
105         // Depth slider
106         JLabel depthLabel = new JLabel("Depth: 10");
107         JSlider depthSlider = new JSlider(1, 15, 10);
108         depthSlider.addChangeListener(e -> {
109           int value = depthSlider.getValue();
110           depthLabel.setText("Depth: " + value);
111           tree.setMaxDepth(value);
112         });
113
114         // Angle slider
115         JLabel angleLabel = new JLabel("Angle: 25°");
116         JSlider angleSlider = new JSlider(10, 45, 25);
117         angleSlider.addChangeListener(e -> {
118           int value = angleSlider.getValue();
119           angleLabel.setText("Angle: " + value + "°");
120           tree.setAngleSpread(value);
121         });
122
123         // Length factor slider
124         JLabel lengthLabel = new JLabel("Length: 0.7");
125         JSlider lengthSlider = new JSlider(50, 90, 70);
126         lengthSlider.addChangeListener(e -> {
127           double value = lengthSlider.getValue() / 100.0;
128           lengthLabel.setText("Length: " + String.format("%.2f", value));
129           tree.setLengthFactor(value);
130         });
131
132         // Add controls to panel
133         controlPanel.add(new JLabel("Recursion "));
134         controlPanel.add(depthSlider);
135         controlPanel.add(depthLabel);
```

```
136
137         controlPanel.add(new JLabel(" | Branch Angle "));
138         controlPanel.add(angleSlider);
139         controlPanel.add(angleLabel);
140
141         controlPanel.add(new JLabel(" | Shrink Factor "));
142         controlPanel.add(lengthSlider);
143         controlPanel.add(lengthLabel);
144
145         // Setup frame
146         frame.setLayout(new BorderLayout());
147         frame.add(tree, BorderLayout.CENTER);
148         frame.add(controlPanel, BorderLayout.SOUTH);
149         frame.pack();
150         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
151         frame.setLocationRelativeTo(null);
152         frame.setVisible(true);
153       });
154     }
155 }
```

6. Task 6: Sudoku Solver with Backtracking (Optional)

Create a Sudoku solver using recursive backtracking. This task builds on the maze pathfinding concepts and applies them to constraint satisfaction problems.

Implement a program that can solve 9x9 Sudoku puzzles using recursive backtracking:

- Read a partially filled Sudoku grid (use 0 for empty cells)
- Find empty cells and try numbers 1-9
- Use recursion to explore all possible solutions
- Backtrack when constraints are violated
- Display the solved puzzle

Your program should:

1. Represent the Sudoku grid as a 2D integer array
2. Implement `isValid(int[][] grid, int row, int col, int num)` to check constraints
3. Create a recursive `solveSudoku(int[][] grid)` method
4. Handle backtracking when no valid numbers can be placed
5. Display the completed puzzle

6.1. Requirements

- Check row, column, and 3x3 box constraints
- Use recursive backtracking algorithm
- Handle cases with no solution
- Display the grid in a readable format

6.2. Assistance

Sudoku grid representation:

```java
public class SudokuSolver {
    private static final int SIZE = 9;
    private static final int EMPTY = 0;

    private int[][] grid = {
        {5, 3, 0, 0, 7, 0, 0, 0, 0},
        {6, 0, 0, 1, 9, 5, 0, 0, 0},
        {0, 9, 8, 0, 0, 0, 0, 6, 0},
        {8, 0, 0, 0, 6, 0, 0, 0, 3},
        {4, 0, 0, 8, 0, 3, 0, 0, 1},
        {7, 0, 0, 0, 2, 0, 0, 0, 6},
        {0, 6, 0, 0, 0, 0, 2, 8, 0},
        {0, 0, 0, 4, 1, 9, 0, 0, 5},
        {0, 0, 0, 0, 8, 0, 0, 7, 9}
    };
}
```

Constraint checking:

```java
private boolean isValid(int[][] grid, int row, int col, int num) {
```

```
    // Check row
    for (int x = 0; x < SIZE; x++) {
        if (grid[row][x] == num) return false;
    }

    // Check column
    for (int x = 0; x < SIZE; x++) {
        if (grid[x][col] == num) return false;
    }

    // Check 3x3 box
    int startRow = row - row % 3;
    int startCol = col - col % 3;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (grid[i + startRow][j + startCol] == num) return false;
        }
    }
    return true;
}
```

6.3. Solution

Java

```
public class SudokuSolver {
  private static final int SIZE = 9;
  private static final int EMPTY = 0;

  // Sample Sudoku puzzle (0 represents empty cells)
  private int[][] grid = {
      {5, 3, 0, 0, 7, 0, 0, 0, 0},
      {6, 0, 0, 1, 9, 5, 0, 0, 0},
      {0, 9, 8, 0, 0, 0, 0, 6, 0},
      {8, 0, 0, 0, 6, 0, 0, 0, 3},
      {4, 0, 0, 8, 0, 3, 0, 0, 1},
      {7, 0, 0, 0, 2, 0, 0, 0, 6},
      {0, 6, 0, 0, 0, 0, 2, 8, 0},
      {0, 0, 0, 4, 1, 9, 0, 0, 5},
      {0, 0, 0, 0, 8, 0, 0, 7, 9}
  };

  // Print the grid
  public void printGrid() {
    for (int row = 0; row < SIZE; row++) {
      if (row % 3 == 0 && row != 0) {
```

```
22          System.out.println("------+-------+------");
23        }
24
25        for (int col = 0; col < SIZE; col++) {
26          if (col % 3 == 0 && col != 0) {
27            System.out.print("| ");
28          }
29
30          if (grid[row][col] == EMPTY) {
31            System.out.print(". ");
32          } else {
33            System.out.print(grid[row][col] + " ");
34          }
35        }
36        System.out.println();
37      }
38    }
39
40    // Check if placing num at grid[row][col] is valid
41    private boolean isValid(int row, int col, int num) {
42      // Check row
43      for (int x = 0; x < SIZE; x++) {
44        if (grid[row][x] == num) {
45          return false;
46        }
47      }
48
49      // Check column
50      for (int x = 0; x < SIZE; x++) {
51        if (grid[x][col] == num) {
52          return false;
53        }
54      }
55
56      // Check 3x3 box
57      int startRow = row - row % 3;
58      int startCol = col - col % 3;
59      for (int i = 0; i < 3; i++) {
60        for (int j = 0; j < 3; j++) {
61          if (grid[i + startRow][j + startCol] == num) {
62            return false;
63          }
64        }
```

```
65      }
66
67      return true;
68    }
69
70    // Find next empty cell
71    private boolean findEmptyCell(int[] cell) {
72      for (int row = 0; row < SIZE; row++) {
73        for (int col = 0; col < SIZE; col++) {
74          if (grid[row][col] == EMPTY) {
75            cell[0] = row;
76            cell[1] = col;
77            return true;
78          }
79        }
80      }
81      return false; // No empty cell found
82    }
83
84    // Solve Sudoku using backtracking
85    public boolean solve() {
86      int[] cell = new int[2];
87
88      // Find next empty cell
89      if (!findEmptyCell(cell)) {
90        return true; // Puzzle solved
91      }
92
93      int row = cell[0];
94      int col = cell[1];
95
96      // Try numbers 1 through 9
97      for (int num = 1; num <= 9; num++) {
98        if (isValid(row, col, num)) {
99          // Place the number
100         grid[row][col] = num;
101
102         // Recursively try to solve the rest
103         if (solve()) {
104           return true;
105         }
106
107         // Backtrack if this number doesn't lead to solution
```

```
108           grid[row][col] = EMPTY;
109         }
110       }
111
112       // No valid number found, trigger backtracking
113       return false;
114     }
115
116     // Solve and display
117     public void solveAndDisplay() {
118       System.out.println("=== Sudoku Solver ===\n");
119       System.out.println("Original puzzle:");
120       printGrid();
121
122       System.out.println("\nSolving...\n");
123
124       if (solve()) {
125         System.out.println("Solution found:");
126         printGrid();
127       } else {
128         System.out.println("No solution exists for this puzzle.");
129       }
130     }
131
132     public static void main(String[] args) {
133       SudokuSolver solver = new SudokuSolver();
134       solver.solveAndDisplay();
135     }
136 }
```

7. Lab Execution

If your program is not yet working without issue, we will try to correct this during the course of the lab. With good preparation, this should not be a problem. Every student is required to be able to explain their thought process at the beginning of the lab. By the end of the lab, the task needs to be completed. Of course, we will support you, but your personal commitment must also be clearly recognizable! Julian Moldenhauer, Furkan Yildirim, and Emily Antosch wish you lots of fun and success!